

Exploring @GeneratedValue and @SequenceGenerator

1. @GeneratedValue Annotation:

- The @GeneratedValue annotation is used to define the strategy for generating primary key values automatically.
- There are different strategies you can use:
 - **GenerationType.IDENTITY**: This strategy lets the database handle the generation of primary keys, commonly used with auto-increment fields.
 - **GenerationType.SEQUENCE**: This strategy is used when a sequence is defined in the database, and Hibernate will use it to generate unique primary key values.
 - **GenerationType.TABLE**: A table is used to maintain the primary key generation.
 - **GenerationType.AUTO**: This allows Hibernate to automatically choose the strategy based on the database used.

Descriptions:

- @GeneratedValue(strategy=GenerationType.IDENTITY) indicates that Hibernate should rely on the auto-increment feature of the database for the primary key.

@Id

@GeneratedValue(strategy = GenerationType.IDENTITY)

private Integer id;

2. @SequenceGenerator Annotation:

- The @SequenceGenerator annotation is used in combination with GenerationType.SEQUENCE to specify a sequence in the database that Hibernate should use to generate primary key values.
- It defines:
 - **name**: The name of the sequence generator.
 - **sequenceName**: The name of the database sequence to use.
 - **initialValue**: The starting value of the sequence.
 - **allocationSize**: Defines the increment size of the sequence. It indicates how many values should be fetched at a time from the database sequence.

Descriptions:

- The custom sequence generator is defined with name="my_seq", which refers to a sequence called "My_OwnSequence", with an initial value of 100 and an allocation size of 1.

```
@GeneratedValue(generator = "my_seq", strategy =
GenerationType.SEQUENCE)
@SequenceGenerator(name = "my_seq", sequenceName =
"My_OwnSequence", initialValue = 100, allocationSize = 1)
private Integer id;
```

Example:

Student.java

```
import jakarta.persistence.Column;
import jakarta.persistence.Entity;
```

```
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.SequenceGenerator;
import jakarta.persistence.Table;
@Entity
@Table(name = "Studenttable")
public class Student {
    @Id
    @GeneratedValue(generator = "my_seq", strategy =
GenerationType.SEQUENCE)
    @SequenceGenerator(name = "my_seq", sequenceName =
"My_OwnSequence", initialValue = 100, allocationSize = 1)
    private Integer id;
    private String name;
    private String city;
    // Zero-argument constructor
    public Student() {
        System.out.println("Zero Param Constructor for Hibernate");
    }
    // Getters and setters
    public Integer getId() {
        return id;
    }
    public void setId(Integer id) {
        this.id = id;
    }
    public String getName() {
        return name;
    }
    public void setName(String name) {
        this.name = name;
    }
    public String getCity() {
```

```

        return city;
    }
    public void setCity(String city) {
        this.city = city;
    }
}

```

Main.java

```

public class Main {
    public static void main(String[] args) {
        Configuration config = null;
        SessionFactory sessionFactory = null;
        Session session = null;
        Transaction transaction = null;
        boolean flag = false;
        try {
            config = new Configuration();
            config.configure();
            sessionFactory = config.buildSessionFactory();
            session = sessionFactory.openSession();
            Student student = new Student();

            student.setName("Arjun");
            student.setCity("Ballia");
            transaction = session.beginTransaction();
            session.persist(student);
            transaction.commit();
            System.out.println("Transaction committed Succesfully");
            flag = true;
        } catch (EntityExistsException e) {
            System.out.println("Entity already exists: " + e.getMessage());
        } catch (ConstraintViolationException e) {

```

```

        System.out.println("Constraint violation: " + e.getMessage());
    } catch (HibernateException e) {
        System.out.println("Hibernate exception: " + e.getMessage());
    } catch (Exception e) {
        System.out.println("Unexpected exception: " + e.getMessage());
    } finally {
        if(!flag)
        {
            if (transaction != null) {
                transaction.rollback();
                System.out.println("Transaction rolled back successfully");
            }
        }
        if (session != null) {
            session.close();
        }
        if (sessionFactory != null) {
            sessionFactory.close();
        }
    }
}
}

```

Note: Never add primary key value if you autoincrement this result in Exception.

Output:

INFO: Hibernate: connection obtained from SubConnectionFactory

Hibernate:
insert into My_OwnSequence values (100)

Hibernate:
create table Studenttable (
id integer not null,
city varchar(255),
name varchar(255),
primary key (id)
) engine=InnoDB

Zero Param Constructor for Hibernate

Hibernate:
select
next_val as id_val
from
My_OwnSequence for update

Hibernate:
update
My_OwnSequence
set
next_val= ?
where
next_val=?

Hibernate:
insert
into
Studenttable
(city, name, id)
values
(?, ?, ?)

Transaction committed Successfully

Sep 15 2024 11:04:50 AM org.hibernate.engine.jdbc.connections

```
mysql> select * from studenttable;
```

id	city	name
100	Ballia	Arjun

1 row in set (0.00 sec)

Summary:

- `@GeneratedValue(strategy=GenerationType.IDENTITY)`: Auto-increments the primary key based on the database's ability to handle the ID generation.
- `@SequenceGenerator`: Used with `GenerationType.SEQUENCE` to specify a custom sequence for generating primary keys. This is helpful when you need more control over the primary key values, especially for databases that use sequences.